

WORK / ODANT-WORK: белая книга

Черновик для накопления понимания архитектуры, продуктовой идеи и системных возможностей платформы.

Назначение

WORK / ODANT-WORK - это платформа для управления организационными процессами через файлово-событийную модель, где чат и звонки являются основным интерфейсом работы, а RAG/ИИ превращает накопленный след деятельности в помощь, аналитику и адаптацию системы.

Главный пользователь системы - организация. В предельном случае это может быть один человек, но базовая модель рассчитана на иерархию подразделений, исполнителей, администраторов и рабочих процессов.

Система не сводится к файловому менеджеру. Файлы здесь являются материализованными событиями деятельности: документами, сообщениями, операциями, письмами, звонками, календарными событиями, результатами работы ИИ и другими артефактами, которые сохраняются, версионизируются, логируются и становятся частью памяти организации.

Три функциональных блока

1. Файлово-объектное ядро.

Базовая модель сущностей: `$folder`, `$file`, `$storage`. Она отвечает за физическое хранение, типизацию, наследование, метапапки, сборку `data.js`, историю, логи и разрешение путей.

1. Системный слой исполнения.

Права доступа, handlers, methods, pages, preview, серверные API, события сохранения, файловые обработчики, RAG-индексация, интеграции и возможность локального переопределения поведения через типизирующие папки.

1. Продуктовый пользовательский слой.

Explorer, дерево структуры, чат, звонки, офисные функции, формы, представления, управление пользователями, работа с подразделениями, ИИ-помощник и будущий

механизм автопрограммирования.

Файловая модель

Система состоит из сущностей трех типов:

- `$folder` - обычная папка.
- `$file` - файл.
- `$storage` - особая папка-хранилище.

Хранилище определяется наличием внутри папки ровно одной метапапки, имя которой начинается с `$`. Имя этой метапапки и есть тип хранилища.

Например, папка с метапапкой `$group` является хранилищем типа `$group`. Папка с метапапкой `$base` является хранилищем типа `$base`.

Метапапка самостоятельно определяет тип. Она может быть не зарегистрирована в общем парке прототипов, но внутри нее предполагается наличие полной цепочки типов от `$storage` до имени самой метапапки в виде реальных или виртуальных вложенных папок.

По этой цепочке сверху вниз "стекает" наследуемая информация: файлы, папки, handlers, настройки, представления и поведение.

Наследование и `data.js`

Для большинства файлов одноименные элементы нижнего уровня перекрывают вышестоящие.

Особый случай - `data.js`. Эти файлы сливаются слой за слоем как код. В серверной реализации `$storage.load()` собирает все доступные `data.js` из наследной ленты, а затем `WorkServer.mergeFiles()` объединяет их через AST-merge.

Это позволяет хранить на разных уровнях не полную копию объекта, а локальное уточнение:

- базовые свойства типа;
- настройки структуры;
- локальные поля;
- права доступа;

- вычисляемые свойства;
- представления и методы.

Организационная модель

Организация моделируется иерархией хранилищ, чаще всего типа `$group` или других кастомных типов.

Каждое подразделение является хранилищем. В его метапапке сохраняются файлы, связанные с деятельностью подразделения: сообщения, документы, операции, письма, звонки, календарные события и другие рабочие артефакты.

Администратор уровня управляет и наблюдает. Назначенные пользователи ведут деятельность и оставляют "хлебные крошки" - файлы, с которыми они работают.

На нижних уровнях исполнители могут принимать роль администратора для своего поддерева. Так возникает управляемая организационная иерархия, где полномочия и рабочий контекст локализованы по подразделениям.

Права и роли

Права администратора проваливаются вниз по организационной иерархии до тех пор, пока на промежуточном уровне не назначен свой администратор. Новый администратор становится основным для нижестоящих подразделений.

Администратор не должен напрямую управлять рабочими файлами в собственной метапапке подразделения. Эти файлы создают назначенные пользователи-исполнители.

Зона администратора - типизирующие папки и наследуемый слой. Через них он может переопределять `data.js`, локальные handlers, представления, методы и другие элементы поведения. На практике это должен делать не сам администратор вручную, а его ИИ-агент с соответствующими правами.

ИИ-агент действует как техническое продолжение администратора, но его ответы и действия должны быть атрибутированы именем модели, например `user: "GigaChat"`. Это нужно для анализа качества, сравнения моделей и восстановления происхождения изменений.

Событие как основная единица процесса

Ключевая архитектурная особенность системы: любое сохранение файла превращается в событие деятельности.

Сценарий выглядит так:

1. Пользователь или handler сохраняет файл через `save_file` или пачку файлов через `save_files`.
2. Хранилище раскладывает файл по назначению внутри метапапки: например `text`, `image`, `logs`, `js`, `etc`.
3. Физический файл сохраняется.
4. Создается копия в `history/<date>/`.
5. Создается запись `data.logs`.
6. Запись попадает в ленту событий, RAG и файловые обработчики.

Таким образом, главный продуктовый объект - не файл сам по себе, а событие деятельности в хранилище, представленное файлом, историей и логом.

На текущем этапе особенно важна надежная регистрация рабочих событий по типам и назначениям файлов, во времени, с пометками пользователей и получателей.

Логи и временной срез

Логи хранят сведения о рабочих событиях:

- время;
- отправитель;
- получатели;
- путь к историческому снимку;
- расширение или тип файла;
- содержимое для сообщений и ответов;
- связанные вложения через `includes`.

Это позволяет рассматривать процесс в разрезе:

- подразделения;
- пользователя;

- периода;
- типа файла;
- получателей;
- цепочки связанных артефактов.

Чат как представление событий

Чат является центральным пользовательским интерфейсом системы.

Важно: чат не является отдельной базой сообщений. Он является представлением над журналом событий хранилища.

Когда пользователь пишет сообщение, оно сохраняется как `message.txt`. Если есть вложения, они сохраняются через `save_files`, а сообщение получает ссылки на них через `includes`.

Ответ ИИ сохраняется как `response.md`. Ошибка сохраняется как `error.txt`. Все это проходит тот же путь: файл, history, log, отображение в chat.

Поэтому чат, аудит, RAG-память и процессная история сходятся в одну модель.

Звонки

Звонки являются вторым базовым интерфейсом взаимодействия. Они должны работать рядом с чатом и также оставлять файлово-событийный след.

Звонок - это не только realtime-коммуникация, но и потенциальное событие процесса: приглашение, факт коммуникации, контекст, участники, результат и дальнейшие артефакты.

RAG и память организации

RAG строится на файлах и логах деятельности. ИИ должен использовать эту память, чтобы понимать процессы, их последовательность, исполнителей, контекст подразделений и накопленные материалы.

RAG должен уметь работать не только в пределах текущего файла или папки, но и по дереву:

- внутри всех папок релевантного хранилища;

- вверх по организационной иерархии хранилищ;
- с учетом локального и вышестоящего контекста.

Граница видимости на текущем этапе должна опираться на права текущего пользователя, но это решение нужно проверять на реальных сценариях.

Через chat ИИ взаимодействует с пользователем, использует RAG-контекст, помогает выполнять процессы, объясняет данные и со временем может брать часть задач на себя.

Handlers, lib и прикладной слой

Функциональность системы наращивается через `lib` и `handlers`.

`lib` предоставляет повторно используемые интерфейсные и системные элементы: explorer, tree, menu, node, security, users, dialogs, file viewer и другие.

`handlers` задают страницы, формы, методы, preview и open-представления для типов объектов.

За счет наследования handlers и lib могут быть переопределены на уровне типа, хранилища или подразделения. Это превращает платформу в систему локально адаптируемого поведения.

Автопрограммирование

Одна из ключевых будущих возможностей - автопрограммирование структур, представлений и методов.

На первом этапе оно должно запускаться только явной командой администратора в чате. Команда должна быть обычной человеческой фразой, потому что пользователи системы - сотрудники, а не разработчики.

Если ИИ распознает запрос на изменение системы, он не должен сразу применять изменение. Сначала он должен прислать администратору план и/или дифф на подтверждение.

После подтверждения ИИ-агент администратора сможет изменять типизирующие папки, `data.js`, handlers и другие наследуемые элементы, оставляя атрибутированный след своих действий.

Эта тема пока зафиксирована как направление развития. До проектирования механизма автопрограммирования нужно полностью описать и стабилизировать текущую

архитектуру.

Базовая поставка

Вокруг ядра должны быть поставлены привычные офисные функции, чтобы организация могла сразу начать работу:

- чат;
- звонки;
- документы;
- календарь;
- email;
- файловые вложения;
- формы и представления;
- управление пользователями и подразделениями;
- ИИ-помощник.

Остальной прикладной функционал должен быть наращиваемым через типы, handlers и локальные расширения.

Рабочая формула

WORK - это платформа для управления организационными процессами через файлово-событийную модель.

Организация представлена иерархией хранилищ. Пользователи создают события через файлы, чат, звонки и офисные действия. Система сохраняет историю и логи. RAG превращает этот след в память организации. ИИ помогает людям работать, анализировать процессы и постепенно адаптировать саму систему под реальные практики подразделений.

Открытые вопросы

- Как именно ранжировать локальный и вышестоящий RAG-контекст.
- Как формализовать границы видимости RAG через права.
- Как описывать назначения файлов помимо расширения и папки назначения.

- Как безопасно проектировать автопрограммирование через чат.
- Как лучше фиксировать действия ИИ-агентов в логах и истории.
- Какие офисные функции входят в минимальную поставку.
- Какие инварианты должны быть покрыты тестами в первую очередь.